Construction and Building Materials 253 (2020) 119208

Contents lists available at ScienceDirect

Construction and Building Materials

journal homepage: www.elsevier.com/locate/conbuildmat

Multi-objective optimization of concrete mixture proportions using machine learning and metaheuristic algorithms

Junfei Zhang [a], Yimiao Huang [a,*], Yuhang Wang [b], Guowei Ma [a,c]

[a] Department of Civil, Environmental and Mining Engineering, The University of Western Australia, Perth 6009, Australia
[b] Institute of Information Engineering, Chinese Academy of Sciences, Beijing 10093, China
[c] School of Civil and Transportation Engineering, Hebei University of Technology, 5340 Xiping Road, Beichen District, Tianjin 300401, China

HIGHLIGHTS

- BPNN has good prediction accuracy for UCS, while RF performs better in predicting slump.
- PSO is efficient in tuning hyperparameters of machine learning models.
- The Pareto front of the mixture optimization problem is obtained by MOPSO.

ARTICLE INFO

Article history:
Received 16 September 2019
Received in revised form 10 April 2020
Accepted 13 April 2020

Keywords:
Concrete
Multi-objective optimization
Machine learning
Particle swarm optimization
Compressive strength
Slump

ABSTRACT

For the optimization of concrete mixture proportions, multiple objectives (e.g., strength, cost, slump) with many variables (e.g., concrete components) under highly nonlinear constraints need to be optimized simultaneously. The current single-objective optimization models are not applicable to multi-objective optimization (MOO). This study proposes an MOO method based on machine learning (ML) and metaheuristic algorithms to optimize concrete mixture proportions. First, the performances of different ML models in the prediction of concrete objectives are compared on data sets collected from the published literature. The winner is selected as the objective function for the optimization procedure. In the optimization step, a multi-objective particle swarm optimization algorithm is used to optimize mixture proportions to achieve optimal objectives. The results show that the backpropagation neural network has better performance on continuous data (e.g., strength), whereas the random forest algorithm has higher prediction accuracy on more discrete data (e.g., slump). The Pareto fronts of a bi-objective mixture optimization problem for high-performance concrete and a tri-objective mixture optimization problem for plastic concrete are successfully obtained by the MOO model. The MOO model can serve as a design guide to facilitate decision-making before the construction phase.

© 2020 Elsevier Ltd. All rights reserved.

1. Introduction

Concrete mixture proportions are conventionally optimized by preparing laboratory trial batches. For instance, to achieve a designed uniaxial compressive strength (UCS), any amount of water, cement, aggregates, and admixtures can be selected to produce concrete samples for testing. This experimental-based method is feasible when there is only one objective (usually UCS) to be optimized. However, in most cases, multiple objectives (e.g., UCS, slump, cost) need to be optimized simultaneously. This will cause an exponential increase in the number of concrete samples to be prepared. Another problem is that experimental-based methods can obtain feasible solutions rather than optimal ones [1].

To address the limitations of these experimental-based methods, computational optimization based on machine learning and metaheuristic algorithms has been used to optimize concrete mixture proportions [1,2]. This optimization method first develops objective functions and then applies metaheuristic optimization algorithms to optimize concrete mixtures. ML models have been applied in this field because they can predict concrete properties without knowing their explicit relationships, thus achieving better prediction accuracy [1]. Several ML models have been widely applied to the prediction of concrete properties, including backpropagation neural networks (BPNNs) [46], support vector regression [47,48], and tree-based models [49–51]. According to the "no free lunch" (NFL) theorem of machine learning, there is no single

* Corresponding author.
E-mail address: yimiao.huang@uwa.edu.au (Y. Huang).

https://doi.org/10.1016/j.conbuildmat.2020.119208
0950-0618/© 2020 Elsevier Ltd. All rights reserved.

ML model that performs universally better than other models for any data set [52]. Therefore, it is necessary to compare the prediction accuracy of several ML models to select the winner as the objective model for the optimization procedure.

After the objective model is established, the next step is to optimize the mixture proportions using an optimization algorithm. Metaheuristic algorithms have been employed as optimization algorithms because they can solve complicated engineering problems with simple codes and high computational efficiency [3,53]. Both metaheuristic algorithms based on particle swarm optimization (PSO) and genetic algorithms can achieve good performance in single-objective mixture optimization [1]. Cheng et al. optimized mixture proportions of high-performance concrete using support vector regression and a genetic algorithm [4]. Their approach can find the minimum cost of a mixture proportion at a specified UCS. Similarly, Behood and Golafshani found the lowest cost of mixture proportions of silica-fume concrete at a specified UCS using biogeography-based programming [5]. Lee also optimized the cost and UCS of high-performance concrete using neural networks and a harmony search algorithm [6]. Yeh designed optimal concrete mixtures using neural networks and optimization technologies and achieved the lowest cost at a given UCS and slump. This method is still a single-objective optimization method and is unable to maximize UCS and slump and minimize the cost of mixtures (i.e., to find the Pareto front of the tri-objective optimization) [7]. The metaheuristic algorithm for single-objective optimization is considerably different from that for multi-objective optimization, and it is not applicable to multi-objective optimization, i.e., to optimize multiple objectives of concrete simultaneously. Only under some special conditions can algorithms suitable for single-objective problems be used for multi-objective optimization (MOO). Pareto optimization should be introduced to find nondominated solutions for MOO problems [8,9].

Therefore, the present study applies multi-objective particle swarm optimization (MOPSO) to optimize concrete mixture proportions. In the following sections, the development of the proposed MOO model will be introduced in Section 2. In Section 3, the MOO model will be applied to two optimization cases: one bi-objective optimization and one tri-objective optimization. Conclusions and future work will be summarized in Section 4.

2. Formulation of the mixture optimization problem

2.1. Modeling objective functions

As stated in the introduction section, ML algorithms are used as objective functions. Particle swarm optimization (PSO) is used to tune the hyperparameters of the ML algorithms.

2.1.1. Machine learning algorithms

2.1.1.1. Backpropagation neural networks. Backpropagation neural networks (BPNNs) are a popular type of artificial neural networks (ANNs). A BPNN includes one input layer, one output layer, and one or more hidden layers. During the training process, the computed output is compared with the actual output. The errors propagate back to adjust the thresholds and weights so as to improve the prediction accuracy of the BPNN [10]. The BPNN topology is plotted in Fig. 1.

The relationship between the input of a neuron and its output is written as

$$O = f\left(\sum_{j=1}^{n}(w_j x_j) + b\right) \tag{1}$$

where O is the output of a neuron, w_j is the weight of an input x_j, b is the bias value of this neuron, and f is the activation function to restrict the output amplitude. The sigmoid activation function is commonly applied as

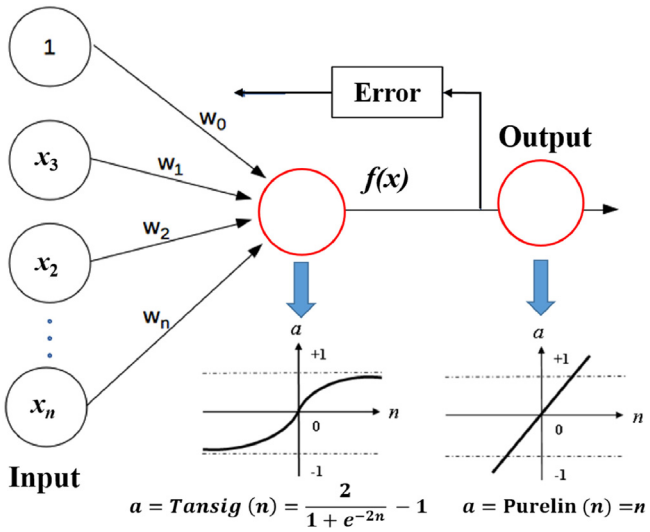


Fig. 1. Backpropagation in a neural network.

$$f(x) = \frac{2}{1 + exp(-x)} - 1 \tag{2}$$

The models perform linearly when operated within nominal parameters (not overdriven). The Purelin transfer function of Matlab (Fig. 1) is an acceptable representation of the input/output behavior in these kinds of situations [11]. In backpropagation, when the mean square error (MSE) between the calculated and actual outputs falls below the defined threshold, the training will stop. The MSE is defined as follows:

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 \tag{3}$$

where y_i and $\hat{y}_i$ denote the actual and calculated output values, respectively.

2.1.1.2. Support vector regression. Support vector regression (SVR) learns the complex relationship between input and output variables by using a kernel to map the data from the sample space into a higher-dimensional character space. SVR is widely employed because it has an outstanding generalization capability, a fast learning speed, and a good noise-tolerating ability [12].

Suppose a training data set of n points is given as follows:

$$\{(\mathbf{x}_1, y_1), (\mathbf{x}_1, y_1), \cdots, (\mathbf{x}_1, y_1)\} \tag{4}$$

where each $\mathbf{x}_i$ is an l-dimensional real director and y_i is the scalar regression value. The regression function can be derived using this data set in the following form:

$$f(\mathbf{x}) = \mathbf{w} \cdot \varphi(\mathbf{x}) + b \tag{5}$$

where $\varphi(\mathbf{x})$ is a nonlinear mapping function, $\mathbf{w}$ represents the weight vector, and b is the bias. Function $f(\mathbf{x})$ is required to be as flat as possible. If for each instance $\mathbf{x}_i$, the deviation between $f(\mathbf{x}_i)$ and y_i is less than ε (the largest tolerance error), the function $f(\mathbf{x}_i)$ is said to be found. A loss function using the ε-insensitive factor is employed to measure the degree of deviation:

$$\mathscr{L}(\mathbf{x}, y, f) = |\mathbf{y}_i - f(\mathbf{x}_i)|_{\varepsilon} = \begin{cases} 0, |\mathbf{y}_i - f(\mathbf{x}_i)| < \varepsilon \\ |\mathbf{y}_i - f(\mathbf{x}_i)| - \varepsilon, |\mathbf{y}_i - f(\mathbf{x}_i)| \geq \varepsilon \end{cases} \tag{6}$$

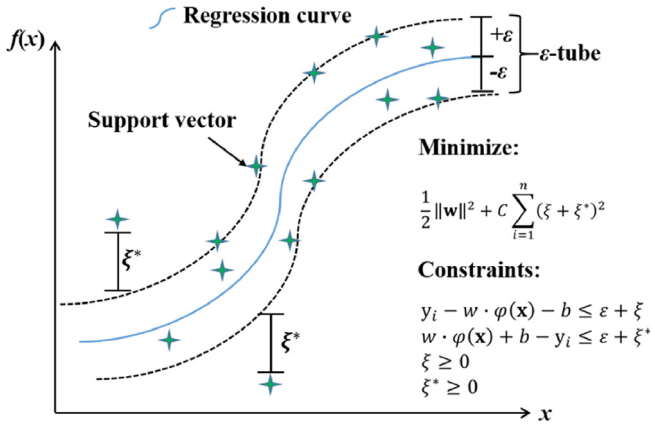


Fig. 2. Example of nonlinear SVR with an ε-tube.

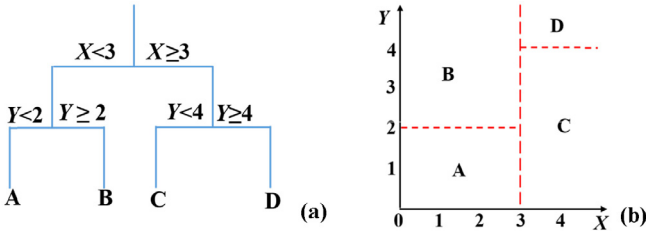


Fig. 3. Regression tree.

This function indicates that the training points within the ε-tube are not penalized and that only the data situated on or outside the ε-tube will be used as support vectors to build $f(\mathbf{x})$. According to structural risk minimization [13], the problem can be written as follows:

$$\mathscr{R}(\mathbf{W}) = \frac{1}{2}\|\mathbf{W}\|^2 + \sum_{i=1}^{n}\mathscr{L}(\mathbf{x}, y, f) \tag{7}$$

Some errors are allowed sometimes, and therefore slack variables ξ_i and ξ_i^* are introduced to cope with infeasible constraints. The above problem can then be converted into the following convex optimization form:

$$min_{\mathbf{w},b,\xi,\xi^*}\mathscr{R}(\mathbf{W}) = \frac{1}{2}\|\mathbf{W}\|^2 + C\sum_{i=1}^{n}(\xi_i + \xi_i^*) \text{ s.t.}$$
$$\begin{cases} y_i - w\cdot\varphi(\mathbf{x}) - b \le \varepsilon + \xi_i \\ w\cdot\varphi(\mathbf{x}) + b - y_i \le \varepsilon + \xi_i^* \\ \xi_i \ge 0 \\ \xi_i^* \ge 0 \end{cases} \tag{8}$$

where C is a penalty parameter to determine the trade-off between the flatness of $f(\mathbf{x})$ and the penalizing extent of the sample outside the tube. An example of nonlinear SVR with an ε-tube is shown in Fig. 2.

To address problems with constraints, Lagrange multipliers can be used as follows:

$$\begin{aligned} L(\mathbf{W}, b, \xi, \alpha, \mu) = {} & \frac{1}{2}\|\mathbf{W}\|^2 + C\sum_{i=1}^{n}(\xi_i + \xi_i^*) \\ & - \sum_{i=1}^{n}\alpha_i(\varepsilon + \xi_i - y_i + \mathbf{W}\cdot\varphi(\mathbf{x_i}) + b) \\ & - \sum_{i=1}^{n}\alpha_i^*(\varepsilon + \xi_i^* + y_i - \mathbf{W}\cdot\varphi(\mathbf{x_i}) - b) \\ & - \sum_{i=1}^{n}(\mu_i\xi_i + \mu_i^*\xi_i^*) \end{aligned} \tag{9}$$

where $\alpha_i \ge 0$, $\alpha_i^* \ge 0$, $\mu_i \ge 0$, and $\mu_i^* \ge 0$ are Lagrange multipliers. When the constraint functions have strong duality and the objective

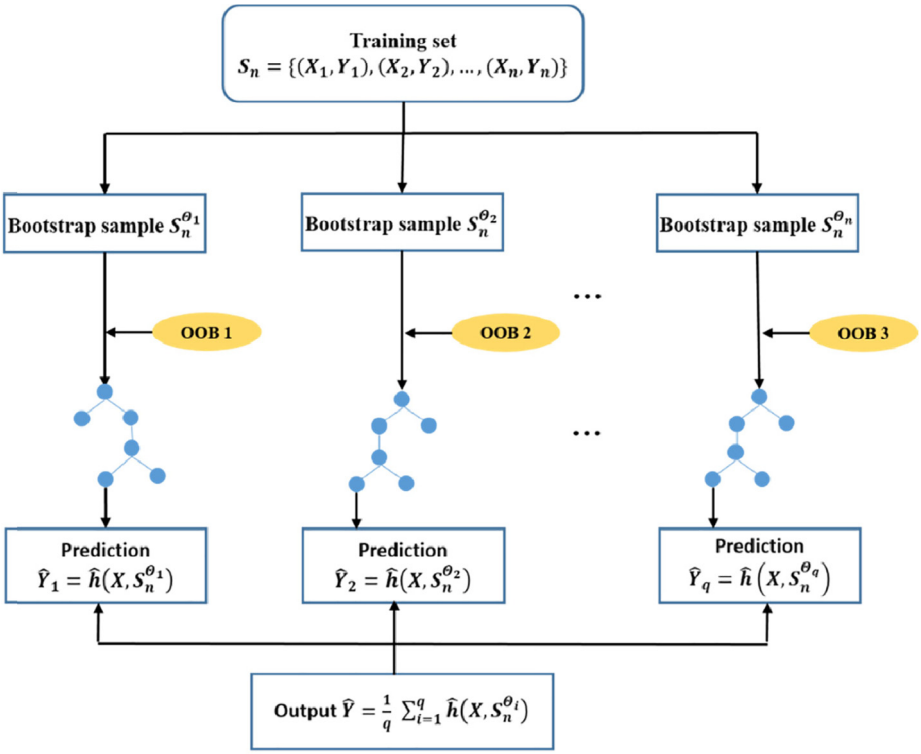


Fig. 4. Construction of an RF.

function is differentiable, Karush-Kuhn-Tucker (KKT) conditions must be satisfied for each pair of the primal and dual optimal points [14], as follows:

$$\begin{cases} \frac{\partial L}{\partial \mathbf{w}} = w - \sum_{i=1}^{n}(\alpha_i - \alpha_i^*)\varphi(\mathbf{x_i}) = 0 \\ \frac{\partial L}{\partial b} = \sum_{i=1}^{n}(\alpha_i - \alpha_i^*) = 0 \\ C - \alpha_i - \mu_i = 0 \\ C - \alpha_i^* - \mu_i^* = 0 \end{cases} \tag{10}$$

In addition, the product between the constraints and the dual variables must be 0 based on the KKT condition at the optimal solution:

$$\begin{cases} \alpha_i(\varepsilon + \xi_i - y_i + \mathbf{w}\cdot\varphi(\mathbf{x_i}) + b) = 0 \\ \alpha_i^*(\varepsilon + \xi_i^* + y_i - w\cdot\varphi(\mathbf{x_i}) - b) = 0 \\ (C - \alpha_i)\xi_i = 0 \\ (C - \alpha_i^*)\xi_i^* = 0 \end{cases} \tag{11}$$

By solving the above equations, the Lagrange dual problem can be derived as follows:

$$\max_i \left(-\frac{1}{2}\sum_{i=1}^{n}\sum_{j=1}^{n}(\alpha_i - \alpha_i^*)\left(\alpha_j - \alpha_j^*\right)\mathbf{x}_i^T\mathbf{x}_j - \varepsilon\sum_{i=1}^{n}(\alpha_i + \alpha_i^*) + \sum_{i=1}^{n} y_i(\alpha_i - \alpha_i^*)\right) \text{s.t.} \begin{cases} \sum_{i=1}^{n}(\alpha_i - \alpha_i^*) = 0 \\ \alpha_i, \alpha_i^* \epsilon [0, C] \end{cases} \tag{12}$$

The weight vector can be obtained as $\mathbf{w} = \sum_{i=1}^{n}(\alpha_i - \alpha_i^*)\varphi(\mathbf{x_i})$, and therefore the regression function can be derived as

$$f(\mathbf{x}) = \sum_{i=1}^{n}(\alpha_i - \alpha_i^*)\varphi(\mathbf{x_i})\mathbf{x} + b \tag{13}$$

2.1.1.3. Regression tree. The regression tree (RT) algorithm splits the sample space into a set of subspaces, and in each subspace a simple RT model is fitted. If we assume that X and Y are two input variables of a regression problem, the sample space is first divided into two subspaces, and in each subspace the output is fitted (Fig. 3a). Then, each subspace is further split, and four new subspaces are formed (see Fig. 3b). The output is fitted in each of the four subspaces. This process continues until the stopping criterion is satisfied. In each split, by selecting the split point and variables, the best fit is obtained. The tree size is defined as the end node count (4 in this example). The output of each subspace is averaged to obtain the final output. The C 4.5 algorithm using "information gain" is widely applied to the split selection process [15], which can be expressed as follows:

$$GainRatio(S, A) = \frac{Gain(S, A)}{SplitInfo(A)} \tag{14}$$

where S is the training set, A is the attribute, and $SplitInfo(A)$ is given by

$$SplitInfo(A) = \sum_{v \epsilon Domain(A)} \frac{\left|S_v^A\right|}{|S|} \cdot log_2 \frac{\left|S_v^A\right|}{|S|} \tag{15}$$

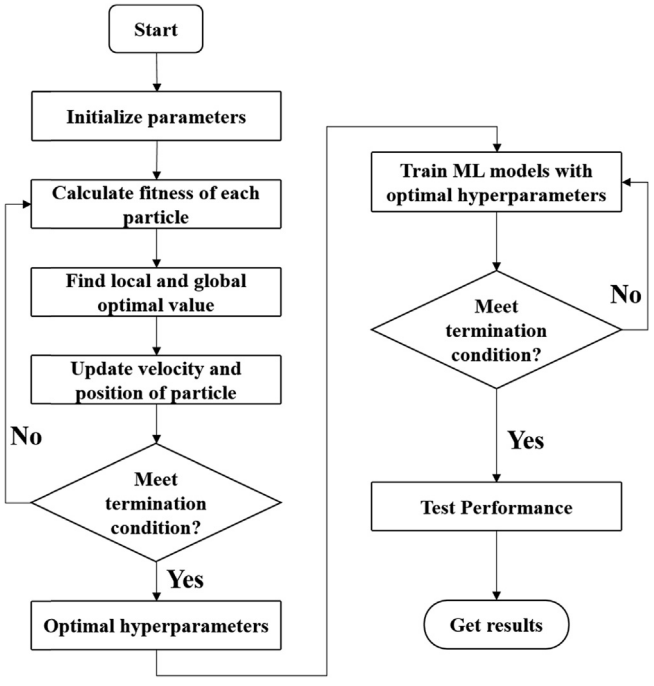


Fig. 5. Hyperparameter tuning using PSO for the ML models.

2.1.1.4. Random forest. A random forest (RF) algorithm generates many decorrelated RTs in the training process. Each tree is grown in a randomly split subset from the training set S_n. The RF then uses a bagging method to combine all the RTs [16]. Bagging can increase the prediction accuracy by reducing the variance related to prediction. Assume that n samples are randomly collected from S_n with a probability of selection of $1/n$ for each sample. These n samples are called a bootstrap sample S_n^{Θ}, where Θ is an independently distributed vector. Assume that q bootstrap samples $(S_n^{\Theta_1}, S_n^{\Theta_2}, \cdots, S_n^{\Theta_q})$ are chosen using the bagging algorithm and that q regression trees are trained on the subsets $\widehat{h}\left(X, S_n^{\Theta_1}\right), \widehat{h}\left(X, S_n^{\Theta_2}\right), \cdots, \widehat{h}\left(X, S_n^{\Theta_q}\right)$. The q outputs are obtained by fitting q RTs: $\widehat{Y}_1 = \widehat{h}\left(X, S_n^{\Theta_1}\right), \widehat{Y}_2 = \widehat{h}\left(X, S_n^{\Theta_2}\right), \cdots, \widehat{Y}_q = \widehat{h}\left(X, S_n^{\Theta_q}\right)$. The final output is obtained by averaging the values of the q outputs. The construction of an RF is demonstrated in Fig. 4.

2.1.1.5. k-nearest neighbor. The k-nearest neighbor (KNN) classifier is based on the idea that the label of a vector $\mathbf{x}$ is determined by the labels of its nearest neighbors. The distance between two input vectors $\mathbf{x}_i$ and $\mathbf{x}_j$ can be defined using the Minkowski metric [17]:

$$\|\mathbf{x}_i - \mathbf{x}_j\|^p = \left(\sum_{i=1}^{q}\left|\mathbf{x}_i - \mathbf{x}_j\right|^p\right)^{1/p} \tag{16}$$

If p = 2, the distance is the Euclidean distance.

The Euclidean distance d_i between $\mathbf{x}$ and each sample is sorted. If d_i ranks in the ith place, then the corresponding sample is called the ith nearest neighbor with output y_i. Finally, the prediction output $\hat{y}$ is the mean of the outputs of its k nearest neighbors, i.e., $\hat{y} = \frac{1}{k}\sum_{i=1}^{k} y_i$.

2.1.1.6. Logistic regression. Assume that $\mathbf{x}$ is a d-dimensional feature vector with class label y = $\{\theta_1, \theta_2\}$. This binary logistic regression (LR) classification problem can be solved as follows [18]:

$$ln\frac{p_1(x)}{1 - p_1(x)} = \beta^T x + \beta_0 \tag{18}$$

where $p_1(x)$ denotes the probability that y = θ_1 and β and β_0 are hyperparameters. Solving the above equation for $p_1(x)$ yields

$$p_1(x) = \frac{1}{1 + exp\left[-\left(\beta^T x + \beta_0\right)\right]} \tag{19}$$

Consider a training set $\{x_i, y_i\}_{i=1}^{n}$, for which β and β_0 can be assessed by maximizing the conditional log-likelihood function

$$\ell(\beta, \beta_0) = \sum_{i=1}^{n} lnp_1(x_i) + (1 - y_{i1})ln[1 - p_1(x_i)] \tag{20}$$

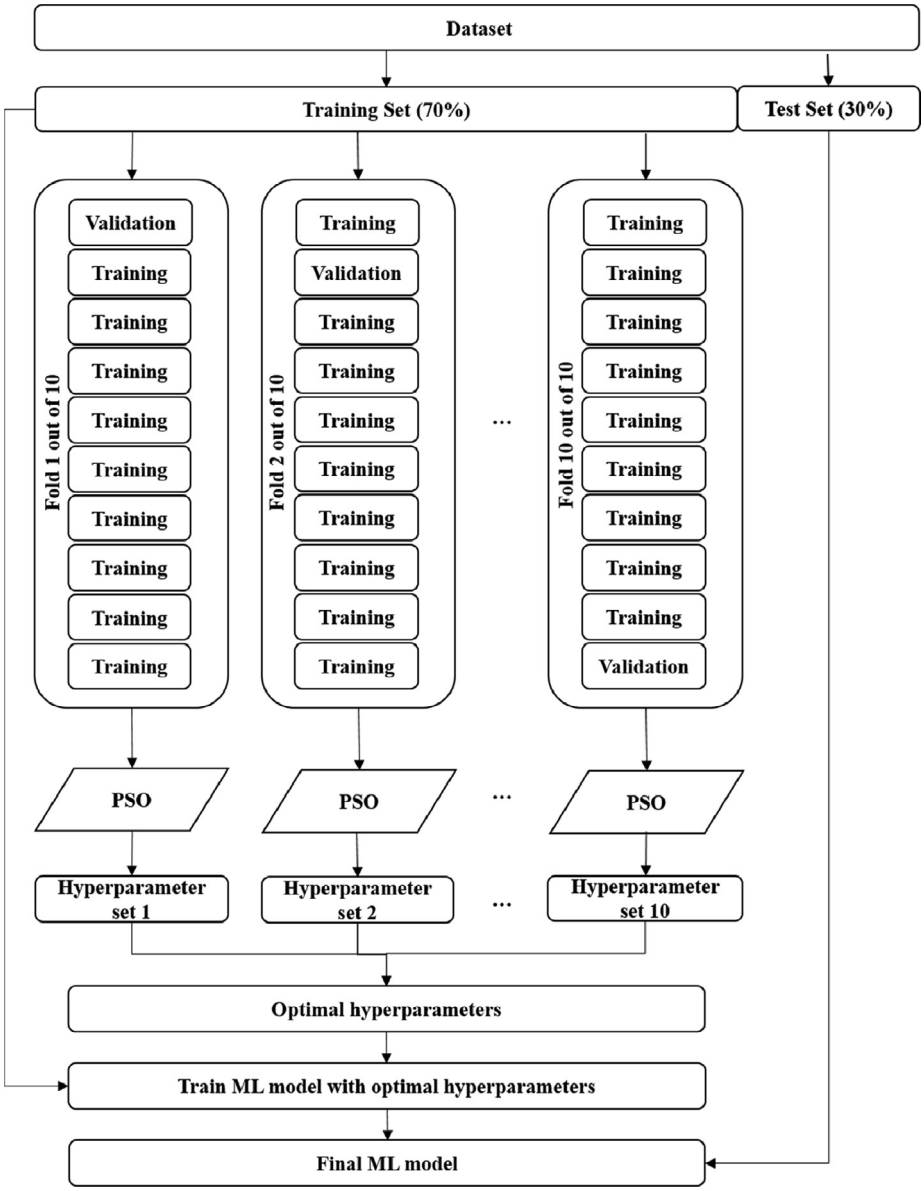


Fig. 6. Hyperparameter tuning using 10-fold cross-validation for the ML models.

Table 1
Hyperparameters of the ML algorithms.

Classifier	Hyperparameter	Empirical scope	Initial value
BPNN	Number of hidden layers (layer_num)	[1,4]	[1,4]
	Number of neurons in each hidden layer (neuron_num)	[1,20]	10
SVR	Coefficient of the penalty term (c)	[0.1, 1000]	16
	Gamma value of the Gaussian kernel (γ)	[0.001, 100]	16
RT	The minimum number of samples required to split an internal node (min_samples_split)	[2,10]	10
	The minimum number of samples required to be at a leaf node (min_samples_leaf)	[1,5]	5
RF	The minimum number of samples required to be at a leaf node (min_samples_leaf)	[1,5]	40
	Total number of trees (tree_num)	[1,50]	40
KNN	Number of neighbor samples (neighbor_num)	[1, 100]	30

where $y_{i,1} = 1$ if $y_i = \theta_1$ and $y_{i,1} = 0$ if $y_i = \theta_2$

2.1.1.7. Multiple linear regression. Multiple linear regression (MLR) can learn the relationships between the input variables and the output variables using the following equation [19]:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_n X_n \quad (21)$$

where X denotes an n-dimensional feature variable and β_i (i = 1, 2, ... n) represents the regression coefficients.

2.1.2. Hyperparameter tuning

Particle swarm optimization (PSO) is applied to tune hyperparameters of the ML algorithms. PSO is a global optimization algorithm that was inspired by the behaviors of bird flocking and fish schooling in a swarm [20]. In PSO, a potential candidate solution is represented by a particle within the searching range. The direction of a particle is adjusted according to its previous best position and the current best position of the other particles, as follows:

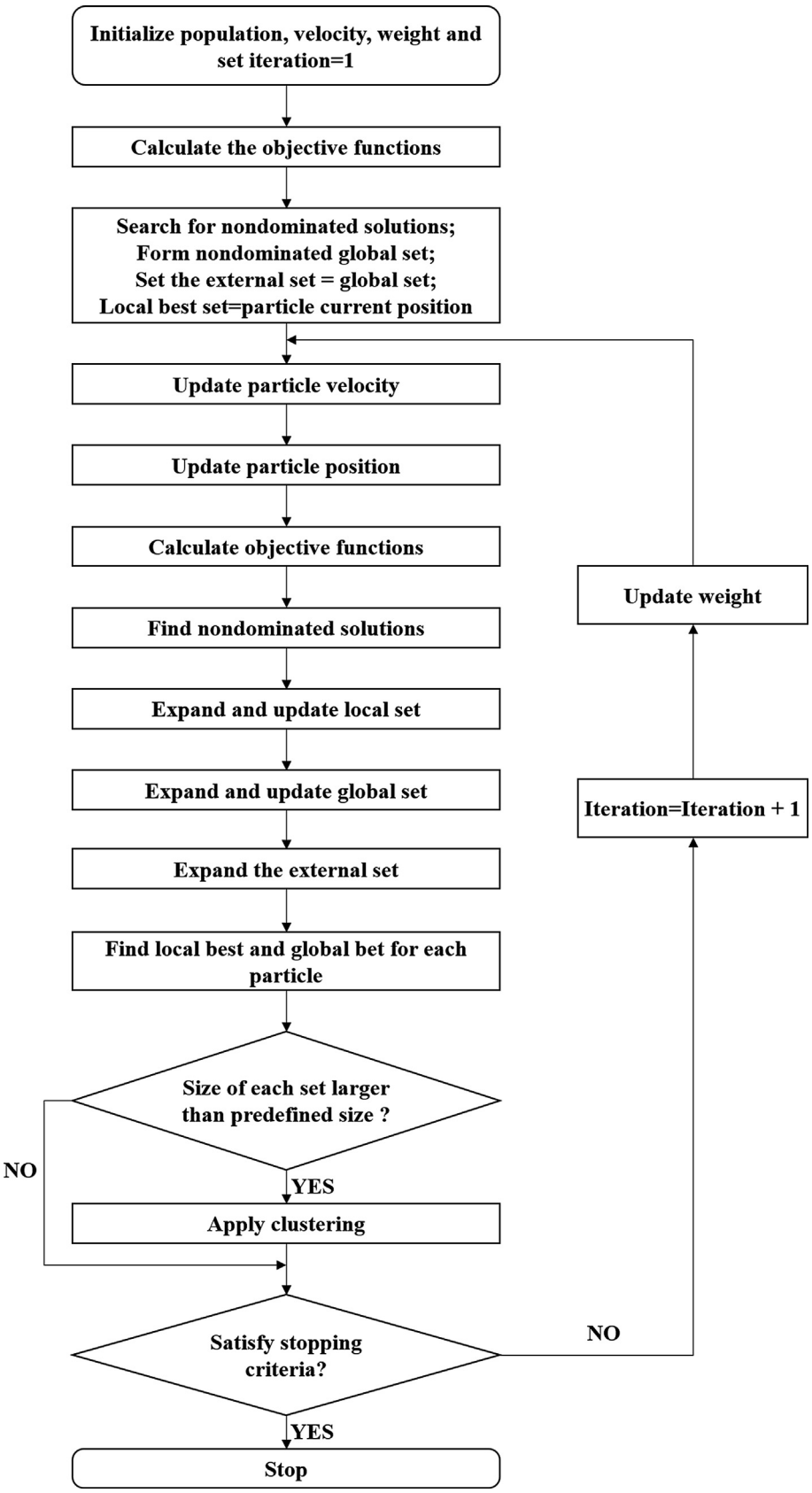


Fig. 7. Flowchart of MOPSO.

$$v_{id}^{t+1} = w \times v_{id}^{t} + c_1 \times r_{1i} \times (pbest_{id} - x_{id}^{t}) + c_2 \times r_{2i} \times (gbest_{id} - x_{id}^{t}) \quad (22)$$

$$x_{id}^{t+1} = x_{id}^{t} + v_{id}^{t+1} \quad (23)$$

where d denotes the dimension of the searching space; x_{id}^{t} and x_{id}^{t+1} are the locations of particle i at the tth and $(t+1)$th iterations, respectively; v_{id}^{t} and v_{id}^{t+1} are the velocities of particle i at the tth and $(t+1)$th iterations, respectively;$pbest_{id}$ and $gbest_{id}$ are the best known position of the particle and the best known position of the entire swarm, respectively;w denotes the initial weights; c_1 and c_2 are acceleration coefficients (usually set to 2); and r_{1i} and r_{2i} represent two random values ranging from 0 to 1. The process of ML hyperparameter tuning by PSO is demonstrated in Fig. 5.

A small data set may cause overfitting issues. This problem can be addressed by introducing 10-fold cross-validation (CV) [21]. In detail, the hyperparameters are tuned on a randomly split training set (the outer training set) including 70% of the instances as per the recommendations in the literature [22]. The outer training data set is further split into an inner training set (including 90% of the outer training data) and a validation set (including 10% of the outer training data). PSO searches for the optimal hyperparameters of the ML algorithms on the inner training set and estimates the model performance by calculating the root-mean-square error (RMSE) on the validation set, as shown in Fig. 6. This process is repeated ten times, and then ten RMSE values are obtained. The hyperparameters of an ML model that have the smallest RMSE are the optimal hyperparameters. The hyperparameters of the five ML algorithms—BPNN, SVM, RT, RF, and KNN—need to be tuned (Table 1).

2.1.3. Evaluation of prediction performance

The following performance measures are applied to assessing the predictive performance of the proposed model.

2.1.3.1. Root-mean-square error (RMSE). RMSE measures the difference between predicted and observed values using the following function [23]:

$$\mathrm{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}\left(y_i^* - y_i\right)^2} \quad (24)$$

where y_i^* is the predicted value, y_i is the actual value, and n is the number of data samples.

2.1.3.2. Correlation coefficient (R). R measures the strength of the correlation between predicted and observed values, which is described as follows [24]:

$$R = \frac{\sum_{i=1}^{n}\left(y_i^* - \overline{y^*}\right)(y_i - \overline{y})}{\sqrt{\sum_{i=1}^{n}\left(y_i^* - \overline{y^*}\right)^2}\sqrt{\sum_{i=1}^{n}(y_i - \overline{y})^2}} \quad (25)$$

where $\overline{y^*}$ and $\overline{y}$ are the mean values of the predicted and observed values, respectively.

2.2. Constraints

To solve the MOO problem, constraints must be set. The constraints can be divided into three categories [5]: (a) range constraints, indicating that the decision variables should vary within a definite range specified by the compiled data set; (b) ratio constraints, stating that several ratios need to be constrained for concrete mixture optimization problems, such as the water-to-cement ratio, coarse aggregate-to-cement ratio, and sand ratio (fine aggregate-to-total aggregate ratio); and (c) the concrete volume constraint, specifying that the total volume of the components in concrete must equal 1 m^3.

2.3. Multi-objective optimization

2.3.1. Definition of the MOO problem

It is relatively easy to find the global optimum of a single-objective optimization problem with one objective function by using simple operators. However, these operators cannot be applied to an MOO problem. The following definitions are commonly used to solve an MOO problem [25].

Definition 1

Minimization problem:

The minimization problem is defined as

$$\min F(\mathbf{x}) = [f_1(\mathbf{x}), f_2(\mathbf{x}), \cdots, f_k(\mathbf{x})]^T$$
$$Subjectto: \begin{cases} g_j(\mathbf{x}) \geq 0, & j = 1, 2, \cdots, t \\ h_j(\mathbf{x}) = 0, & j = 1, 2, \cdots, m \\ l_j \leq x_j \leq \mu_j, & j = 1, 2, \cdots, p \end{cases} \quad (26)$$

where $F(\mathbf{x})$ is the objective function containing k objectives; t, m, and p are the numbers of inequality constraints, equality constraints, and variables, respectively; $g_j(\mathbf{x})$ and $h_j(\mathbf{x})$ are the jth inequality and equality constraints, respectively; and $[l_j, \mu_j]$ denotes the boundaries of the jth variable.

Definition 2

Pareto dominance:

Assume that $\mathbf{u}, \mathbf{v} \in \mathbb{R}^D$ are two vectors. Then, $\mathbf{u}$ dominates $\mathbf{v}$ ($\mathbf{u} \prec \mathbf{v}$) if $\forall i \in \{1, 2, \cdots, D\}$: $u_i \leq v_i$, and $\exists i \in \{1, 2, \cdots, D\}$:$u_i < v_i$.

Definition 3

Pareto set and Pareto front:

For a given MOP F($\mathbf{x}$), assume that $\Gamma \subset S_x$ is a vector set. Then, $\mathbf{x}^* \in \Gamma$ is called a Pareto solution if there is no $\mathbf{x} \in \Gamma$ that satisfies F($\mathbf{x}$) $\prec$ F($\mathbf{x}^*$). The Pareto set is given by

$$P_\Gamma = \{\boldsymbol{x}^* \in \boldsymbol{\Gamma} | \neg\exists \boldsymbol{x} \in \boldsymbol{\Gamma} : F(\boldsymbol{x}) \prec F(\boldsymbol{x}*)\} \quad (27)$$

where $\neg\exists$ represents nonexistence. The Pareto front is defined as

$$PF = \{F(\boldsymbol{x}) \in S_y | \boldsymbol{x} \in P_\Gamma\} \quad (28)$$

Table 2
Statistics of the HPC data set.

Variables	Min	Max	Mean	Median	SD
Cement (kg/m^3)	102.00	540.00	276.50	266.00	103.47
Blast-furnace slag (kg/m^3)	0.00	359.40	74.27	26.00	84.25
Fly ash (kg/m^3)	0.00	260.00	62.81	0.00	71.58
Water (kg/m^3)	121.75	247.00	182.98	185.70	21.71
Superplasticizer (kg/m^3)	0.00	32.20	6.42	6.70	5.80
Coarse aggregate (kg/m^3)	708.00	1145.00	964.83	966.80	82.79
Fine aggregate (kg/m^3)	594.00	992.60	770.49	777.50	79.37
Curing age (day)	1.00	365.00	44.06	28.00	60.44
UCS (MPa)	2.33	82.60	35.84	34.67	16.10

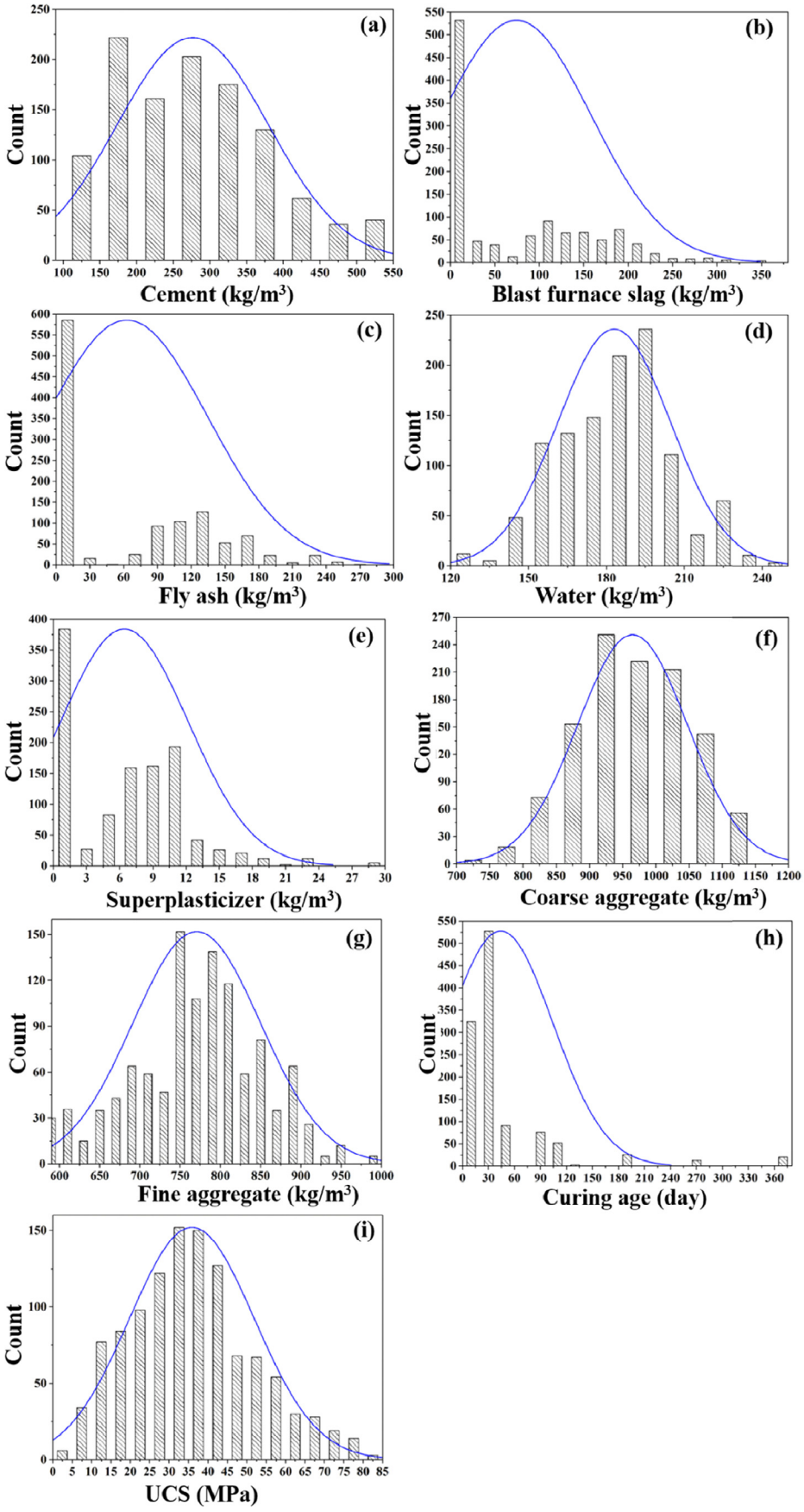


Fig. 8. Histogram of the input and output variables in the HPC data set.

Table 3
Range constraints of the input and output variables.

Component	Unit	Unit weight (kg/m^3)	Cost ($/m^3)	Min ($/m^3)	Max ($/m^3)
Cement	kg/m^3	3150	0.110	102.00	540.00
Blast-furnace slag	kg/m^3	2800	0.060	0.00	359.40
Fly ash	kg/m^3	2500	0.055	0.00	260.00
Water	kg/m^3	1000	0.00024	121.75	247.00
Superplasticizer	kg/m^3	1350	2.940	0.00	32.20
Coarse aggregate	kg/m^3	2500	0.010	708.00	1145.00
Fine aggregate	kg/m^3	2650	0.006	594.00	992.60
UCS (28 d)	MPa	–	–	8.54	81.75

Table 4
Ratio constraints of the mixture parameters.

Ratio	Expression	Lower bound	Upper bound
Water to binder	$C_W/(C_C + C_{BFS} + C_{FLA})$	0.23	0.9
Fly ash to binder	$C_{FLA}/(C_C + C_{BFS} + C_{FLA})$	0	0.61
Blast-furnace slag to binder	$C_{BFS}/(C_C + C_{BFS} + C_{FLA})$	0	0.61
Fine aggregate to total aggregate	$C_{CA}/(C_{FA} + C_{CA})$	0.35	0.54
Coarse aggregate to binder	$C_{CA}/(C_C + C_{BFS} + C_{FLA})$	1.18	5.62
Superplasticizer to cement	$C_{SP}/(C_C)$	0	0.13

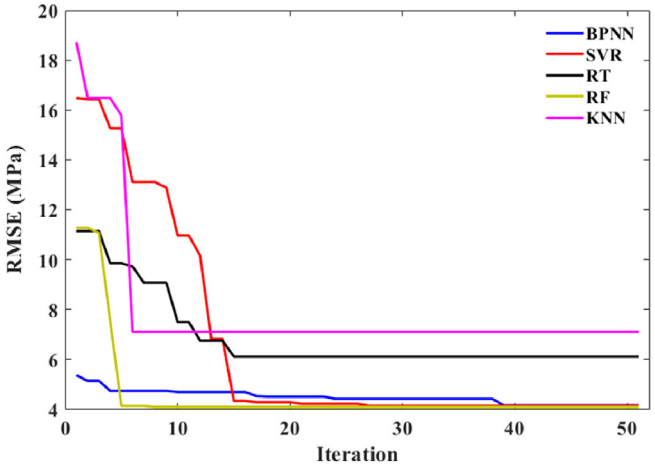


Fig. 9. RMSE versus iteration on the validation set during hyperparameter tuning for HPC.

Definition 4

Pareto optimal set:

For a given MOP F(**x**), $\mathbf{x}^* \subset S_x$ is a Pareto optimal solution if there exists no feasible solution x satisfying $F(\mathbf{x}) \prec F(\mathbf{x}^*)$. The Pareto optimal set $\wedge$ is defined as

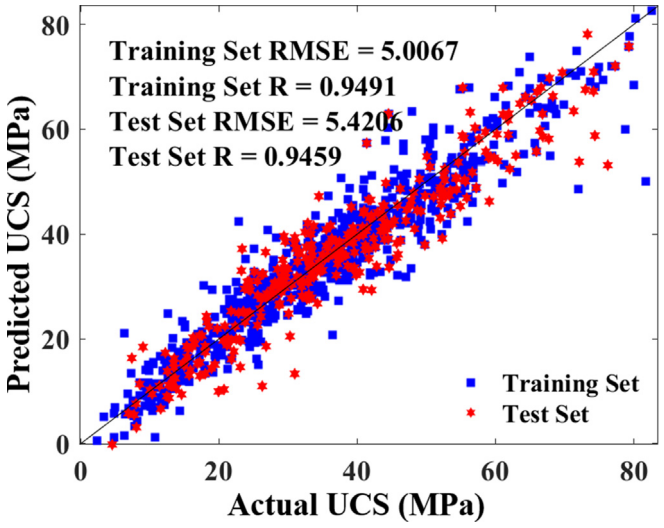


Fig. 10. Predicted versus actual UCS values on the training and test sets using BPNN for HPC.

$$\wedge = \{\boldsymbol{x}^* \in S_x | \neg \exists \boldsymbol{x} \in S_x : F(\boldsymbol{x}) \prec F(\boldsymbol{x}*)\} \quad (29)$$

Definition 5

Pareto optimal front:

For a given MOP F(**x**) and a Pareto optimal set $\wedge$, the Pareto optimal front is defined as

$$PF^* = \{F(\boldsymbol{x}) \in S_y | \boldsymbol{x} \in \wedge\} \quad (30)$$

2.3.2. Multi-objective particle swarm optimization

Traditionally, multi-objective problems are solved by converting multiple objectives into a single objective using the ε-constraint method [26] or weighted sum method [27]. These methods will run several times before the best parameter setting is achieved and thus are computationally inefficient. Furthermore, for disjointed and concave Pareto fronts, these methods cannot achieve accurate results. These issues can be accommodated by multi-objective particle swarm optimization (MOPSO).

Table 5
Optimal hyperparameters of the ML algorithms for HPC.

Model	Hyperparameter	RMSE (MPa)		R	
		Training set	Test set	Training set	Test set
BPNN	Layer_num = 1				
	neuron_num = 12	5.0	5.4	0.95	**0.95**
SVR	C = 61; γ = 6.3	3.4	5.5	0.98	0.94
RT	min_samples_split = 2; min_samples_leaf = 1	5.4	7.8	0.94	0.88
RF	min_samples_leaf = 1;tree_num = 10	2.7	5.4	0.99	**0.95**
KNN	neighbor_num = 3	2.5	9.4	0.99	0.84
LR	–	12.3	12.2	0.77	0.77
MLR	–	9.8	10.5	0.79	0.77

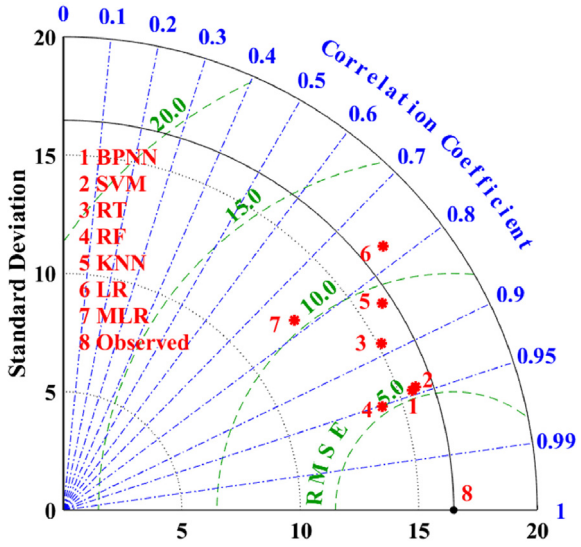


Fig. 11. Comparison of prediction accuracy of the ML algorithms using the Taylor diagram for HPC.

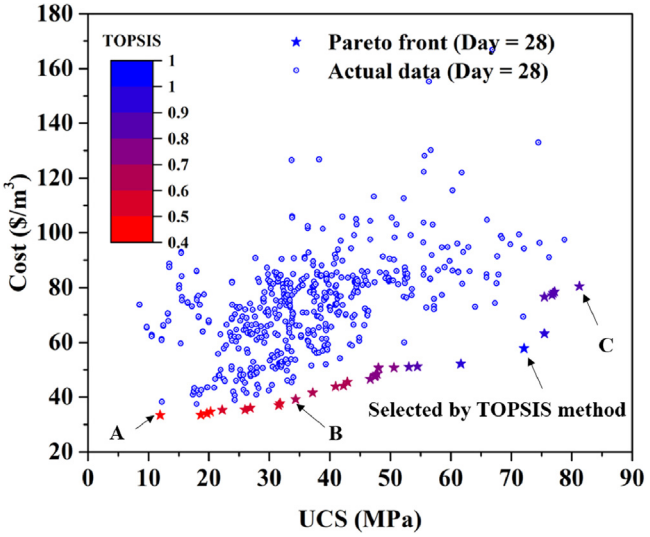


Fig. 12. Pareto front showing the trade-off between cost and 28-day UCS for HPC.

As described before, PSO is efficient in dealing with single-objective optimization problems. MOPSO was developed based on single PSO to solve MOO problems [28]. Like PSO, MOPSO finds the global optimization position based on the best known position of the individual particle and the particle swarm. Nonetheless, in MOO problems, several global optima exist that constitute the Pareto front. Therefore, the MOPSO collects all the nondominated particles into a repository, and then each particle selects its leader in the repository based on the adaptive grid method. Finally, a diverse Pareto front is obtained (Fig. 7).

2.4. Decision-making for multi-objective optimization problems

Concrete mixture optimization is an MOO problem, and hence it is necessary to introduce a multi-criteria decision analysis approach to obtain a final optimal solution from the Pareto front. To this end, the technique for order preference by similarity to an ideal solution (TOPSIS) is widely used [29]. This method selects an alternative that is closest to the positive ideal point and farthest from the negative ideal point. In the Pareto optimal front, the negative ideal point is defined as the solution with the worst objective function value, while the positive ideal point is the solution with the best value of the objective function:

$$d_{i+} = \sqrt{\sum_{j=1}^{n}\left(F_{ij} - F_j^{ideal}\right)^2} \tag{31}$$

$$d_{i-} = \sqrt{\sum_{j=1}^{n}\left(F_{ij} - F_j^{non-ideal}\right)^2} \tag{32}$$

where d_{i+} and d_{i-} are the positive ideal point and the negative ideal point, respectively; n represents the number of objectives; i denotes a solution point in the Pareto front; and F_j^{ideal} and $F_j^{non-ideal}$ stand for the ideal and nonideal values for the jth objective in a single-objective optimization, respectively. The closeness coefficient is defined as follows:

$$C_i = \frac{d_{i-}}{d_{i+} + d_{i-}} \tag{33}$$

A solution with the highest C_i on the Pareto front is selected as the final optimal solution.

3. Case study

In this section, two cases are discussed: a bi-objective mixture optimization problem for high-performance concrete (HPC) and a tri-objective mixture optimization problem for plastic concrete.

3.1. Case 1: bi-objective optimization

In the design of HPC mixtures, UCS is one of the basic mechanical parameters to be considered in design codes and standards. Another key consideration for proportioning HPC mixtures is that the cost must be as low as possible without compromising the UCS. The economic implications can be enormous in projects that require large amounts of concrete. Therefore, this study optimizes

Table 6
Mixture proportions of different selected points on the Pareto front for HPC.

Mixture	A	B	C	Final point
Cement (kg/m^3)	102	117.8	504.3	329.9
BFS (kg/m^3)	0	143.4	217.7	180.0
FLA (kg/m^3)	125.4	38.0	45.9	64.1
Water (kg/m^3)	164.9	121.8	178.7	202.8
SP (kg/m^3)	0	0	0	0.50
CA (kg/m^3)	1005.9	1100.3	973.2	1100.0
FA (kg/m^3)	975.0	969.9	594	928.4
UCS (MPa)	18.7	34.3	81.3	72.1
Cost ($/m^3)	33.5	39.3	80.5	57.8
TOPSIS score	0.50	0.60	0.87	1

Table 7
Statistics of the variables in the slump data set.

Variables	Min	Max	Mean	Median	SD
Water (kg/m^3)	152.1	520	343.1	335	68.5
Cement (kg/m^3)	72	300	181.2	200	50.2
Gravel (kg/m^3)	0	926	690.2	774.5	211.2
Sand (kg/m^3)	441	1499	811	750	206.6
Silty clay (kg/m^3)	0	260	33.6	0	75.1
Bentonite (kg/m^3)	15	168	45.0	39	27
Slump (mm)	8	240	179.1	190	48.8

Table 8
Statistics of the variables in the UCS data set.

Variables	Min	Max	Mean	Median	SD
Water (kg/m^3)	190	500	336	357	77
Cement (kg/m^3)	50	252	134	120	40
Gravel (kg/m^3)	295	875	616	655	192
Sand (kg/m^3)	524	1305	841	662	256
Silty clay (kg/m^3)	0	380	159	180	93
Bentonite (kg/m^3)	16	320	73	63	39
Curing time (day)	7	540	84	28	132
UCS (MPa)	0.8	21.78	3.98	2.48	3.64

the UCS and cost objectives, i.e., to obtain the largest possible UCS with minimum cost.

3.1.1. Data set description

A widely used concrete data set from the machine learning repository of the University of California Irvine (UCI) (http://archive.ics.uci.edu/ml/datasets/Concrete+Compressive+Strength) is used. This data set, which includes 1030 samples of high-performance concrete (HPC), was provided by I-Cheng Yeh [30] (see the Supplementary Materials). This data set includes the following input variables: content of cement (C_C), content of blast-furnace slag (C_{BFS}), content of fly ash (C_{FLA}), content of water (C_W), content of fine aggregate (C_{FA}), content of coarse aggregate (C_{CA}), content of superplasticizer (C_{SP}), and curing age (Age). The output variable is UCS of HPC. The statistics of the input and output parameters are tabulated in Table 2. The histogram of the input and output variables is shown in Fig. 8.

3.1.2. Problem formulation for the bi-objective optimization problem

3.1.2.1. Bi-objective function. The UCS of HPC is predicted using PSO-tuned ML models. The cost objective is calculated using a polynomial equation [5,31,32]:

$$\text{Cost} = C_C Q_C + C_{BFS} Q_{BFS} + C_{FLA} Q_{FLA} + C_W Q_W + + C_{SP} Q_{SP} + C_{FA} Q_{FA} + C_{CA} Q_{CA} \quad (34)$$

where C_C, C_{BFS}, C_{FLA}, C_W, C_{SP}, C_{FA}, and C_{CA} are the unit prices of cement (0.11 \$/kg), blast-furnace slag (0.060 \$/kg), fly ash (0.055 \$/kg), water (0.00024 \$/kg), superplasticizer (2.94 \$/kg), fine aggregate (0.006 \$/kg), and coarse aggregate (0.010 &/kg), respectively (Table 3). The variables Q_C, Q_{BFS}, Q_{FLA}, Q_W, Q_{SP}, Q_{FA}, and Q_{CA} denote the quantities of cement, blast-furnace slag, fly ash, water, superplasticizer, fine aggregate, and coarse aggregate, respectively.

3.1.2.2. Constraints. As mentioned earlier, range constraints, ratio constraints, and volume constraints are set. The range and ratio constraints are listed in Tables 3 and 4, respectively. The concrete volume constraint is given by

$$V_m = \frac{C_C}{U_C} + \frac{C_{BFS}}{U_{BFS}} + \frac{C_{FLA}}{U_{FLA}} + \frac{C_W}{U_W} + \frac{C_{SF}}{U_{SF}} + \frac{C_{FA}}{U_{FA}} + \frac{C_{CA}}{U_{CA}} + \frac{C_{SP}}{U_{SP}} = 1\ \text{m}^3 \quad (35)$$

where U_C, U_{BFS}, U_{FLA}, U_W, USF, U_{FA}, U_{CA}, and U_{SP} are the unit weights of cement, blast-furnace slag, fly ash, water, silica fume, fine aggregate, coarse aggregate, and superplasticizer, respectively (see Table 3).

3.1.3. Performance of the ML algorithms in predicting UCS of HPC

As mentioned above, PSO is used to search for optimal hyperparameters of the ML algorithms. The RMSE versus iteration curves for the ML algorithms on the validation set at the best fold are plotted in Fig. 9. It is evident that the RMSE decreases significantly with iteration, indicating that PSO is efficient in tuning hyperparameters of ML algorithms. After convergence, the optimal hyperparameters are obtained (Table 5). The performance of the ML models with optimal hyperparameters is then evaluated on both training and test sets. Figure 10 shows the scatter plot of actual UCS versus predicted UCS on the training and test sets. Table 5 shows the achieved R and RMSE values on training and test sets for all the ML models. The highest R values on test set are typed in bold. It can be observed that on the test set, BPNN and RF achieve the highest R value (0.95), which is slightly higher than that obtained by SVR (0.94). This indicates that all these three models can accurately capture the complex relationship between UCS and the components of HPC. This pattern can also be obtained from the Taylor diagram (Fig. 11), which graphically indicates which of the models is most realistic by measuring the distance between each model and the point labeled "observed" (the closer, the better). The R values of RT and KNN on the test set are smaller than 0.9 (0.88 and 0.84, respectively), suggesting a lower prediction accuracy than that of the three abovementioned ML models. It should be noted that KNN suffers most from overfitting, with the largest overfitting ratio of 3.76 (the training set RMSE divided by the test set RMSE). LR and MLR achieve the lowest RMSE (12.3 MPa and 10.5 MPa, respectively) and R (0.77) on the test set, indicating the lowest prediction accuracy of these two models. BPNN is used as the objective function of UCS of HPC in the optimization process. The generated BPNN model, including the weights and biases of the input, hidden, and output layers, are included in the Supplementary Materials.

3.1.4. Results of bi-objective optimization for HPC

The bi-objective mixture optimization results for HPC are shown in Fig. 12. BPNN is used as the objective function for modeling UCS of HPC. The Pareto front based on cost and 28-day UCS is obtained. In total, 30 nondominated solutions (the optimal mixture proportions) are selected. The values of UCS and cost are widely

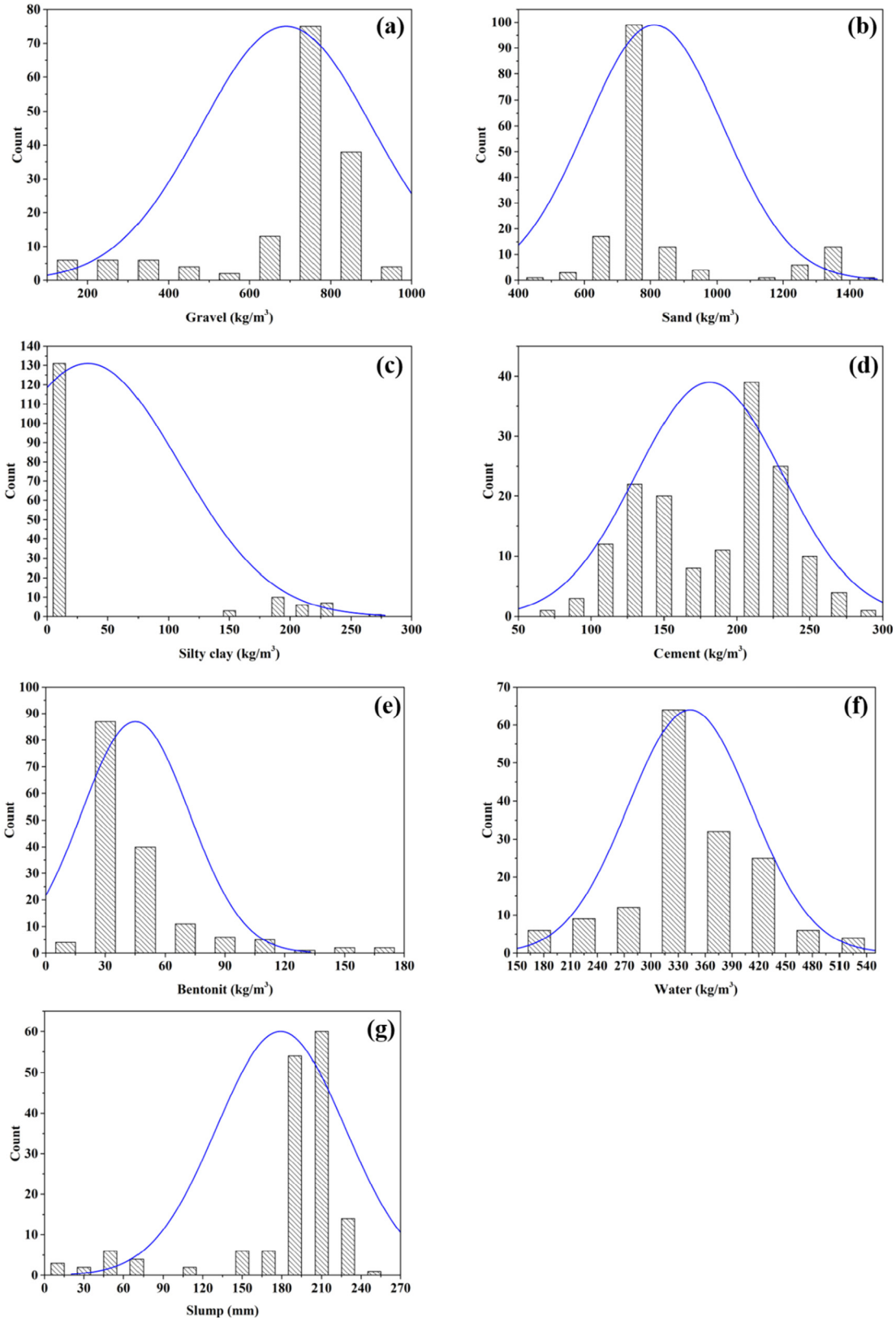


Fig. 13. Histogram of the input and output variables in the slump data set for plastic concrete.

distributed within a reasonable range, indicating that the MOO model has high effectiveness and generalization capabilities. All the mixtures in the data set are located above the Pareto front, suggesting that the cost of each mixture in the data set is higher than necessary for reaching the required strength and that the MOO model is efficient in reducing the cost of the mixtures. It can also

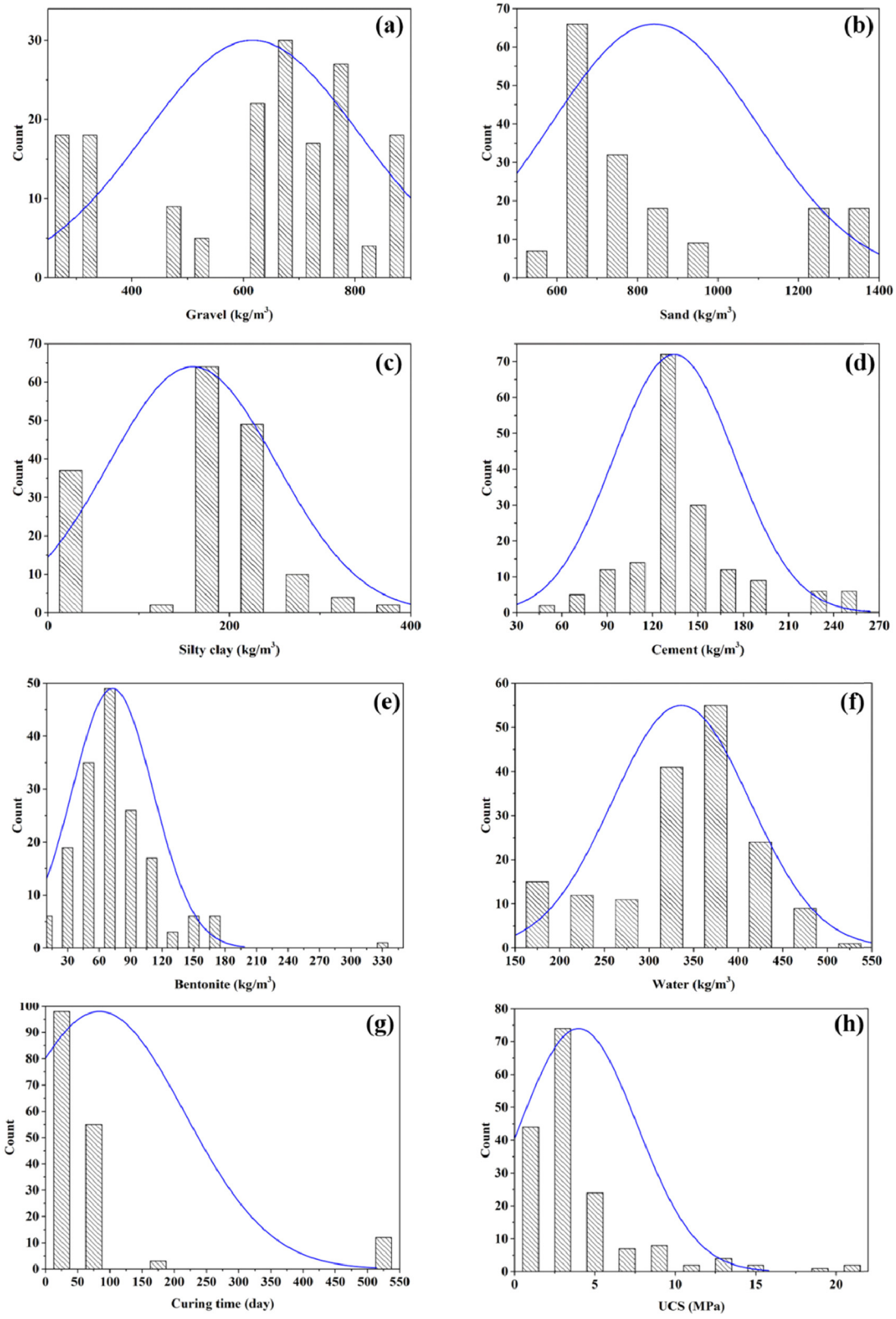


Fig. 14. Histogram of the input and output variables in the UCS data set for plastic concrete.

be observed that to increase the UCS, an increase in the mixture cost is needed to achieve this.

The TOPSIS score (closeness coefficient) obtained for each point on the Pareto front is shown in Fig. 12. It can be seen that the mixture proportion with the highest closeness coefficient (C = 1) has a UCS of 72.1 MPa and a cost of 57.8 \$/m^3. This mixture is then recommended as the final optimal mixture. The selection of optimal mixtures can also be based on engineering requirements. For instance, from Fig. 12, solution C on the Pareto front is more expensive but has a higher UCS, whereas solution A has a lower UCS with

Table 9
Constraints of the parameters.

Parameter	Expression	Lower bound	Upper bound
Cement content (kg/m^3)	C_c	50	300
Water-to-cement ratio	C_w/C_c	0.7	9
Gravel-to-cement ratio	C_g/C_c	0	15
Sand ratio	$C_s/(C_g + C_s)$	0.375	1
Bentonite-to-cement ratio	C_b/C_c	0.072	2.6
Silty clay-to-cement ratio	C_{sc}/C_c	0	4.6
Slump (mm)	Slump	8	240
UCS (MPa)	UCS	1.1	12

a correspondingly lower cost. It also shows that solution B has an intermediate cost and UCS compared with solutions A and C. Note that the optimal solution depends on the mixture design consideration. Solution A is the optimal solution when minimizing cost is the main objective, and solution C will be the best option if the objective is to achieve the maximum UCS. These mixture proportions are tabulated in Table 6.

3.2. Case 2: tri-objective optimization

In many dewatering projects in civil engineering, such as reservoirs and dams, concrete cut-off walls are usually constructed to control water seepage [33]. However, concrete cut-off walls are rigid and may rupture due to the deformation of the earth embankment. Therefore, materials for constructing cut-off walls are supposed to be watertight and strong and to have a stiffness comparable to that of the surrounding soil. These material properties will ensure that the soil and wall deform without separating. To this end, bentonite can be added to the concrete mixture to increase concrete ductility. This kind of concrete is called plastic concrete and satisfies the permeability, stiffness, and strength requirements for constructing cut-off walls [34]. Plastic concrete has comparatively lower strength and permeability but higher formability due to the use of bentonite clay with a higher water-to-cement (w/c) ratio [35].

3.2.1. Data set description

It is known that the four basic components of concrete are cement, water, fine aggregate (sand in this study), and coarse aggregate (gravel in this study) [36]. As stated before, plastic concrete is supposed to be more ductile than conventional concrete, and thus bentonite and silty clay are incorporated in the mixture to produce a ductile material [35]. Based on these influencing input variables, a comprehensive data set was collected from the international literature, including 149 different mixtures for slump and 168 different mixtures for UCS at 28 days [37–44] (see the Supplementary Materials). The statistics of the slump data set and UCS data set are tabulated in Tables 7 and 8, respectively. The histograms of the input and output variables in the slum and UCS data sets are plotted in Figs. 13 and 14, respectively.

3.2.2. Problem formulation for the tri-objective optimization

The range constraints and ratio constraints used for the mixture optimization of plastic concrete are listed in Table 9. The concrete volume constraint is given by

$$V_m = \frac{Q_c}{U_c} + \frac{Q_w}{U_w} + \frac{Q_s}{U_s} + \frac{Q_g}{U_g} + \frac{Q_b}{U_b} + \frac{Q_{sc}}{U_{sc}} = 1\ \mathrm{m}^3 \quad (36)$$

where Q_c, Q_w, Q_s, Q_g, Q_b, and Q_{sc} denote the weights of cement, water, sand, gravel, bentonite, and silty clay, respectively. U_c (3150 kg/m^3), U_w (1000 kg/m^3), U_s (2650 kg/m^3), U_g. (2500 kg/m^3), U_b (2200 kg/m^3), and U_{sc} (1330 kg/m^3) are the unit weights of cement, water, sand, gravel, bentonite, and silty clay, respectively.

The objective functions for UCS and slump are modeled using the ML models. The objective function of mixture cost is calculated by polynomial equations, as follows [5,31,32]:

$$\mathrm{Cost} = C_cQ_c + C_wQ_w + +C_sQ_s + C_gQ_g + C_bQ_b + C_{sc}Q_{sc} \quad (37)$$

where C_c, C_w, C_s, C_g, C_b, and C_{sc} are the unit prices of cement (0.11 \$/kg), water (0.00024 \$/kg), sand (0.006 \$/kg), gravel (0.01 \$/kg), bentonite (0.077 \$/kg), and silty clay (0.013 \$/kg), respectively. The variables Q_c, Q_w, Q_s, Q_g, Q_b, and Q_{sc} are the quantities of cement, water, sand, gravel, bentonite, and silty clay, respectively, per cubic meter.

3.2.3. Performance of the ML algorithms in predicting the UCS and slump of plastic concrete

Fig. 15 shows the architecture tuning process on the training set of UCS (Fig. 15a) and slump (Fig. 15b), respectively. It can be seen that for both slump and UCS prediction, all the RMSE curves converge within 50 iterations, indicating that PSO is efficient in finding the optimal hyperparameters of the ML models. It should be noted that for some models (e.g., BPNN in Fig. 15a and KNN in Fig. 15b),

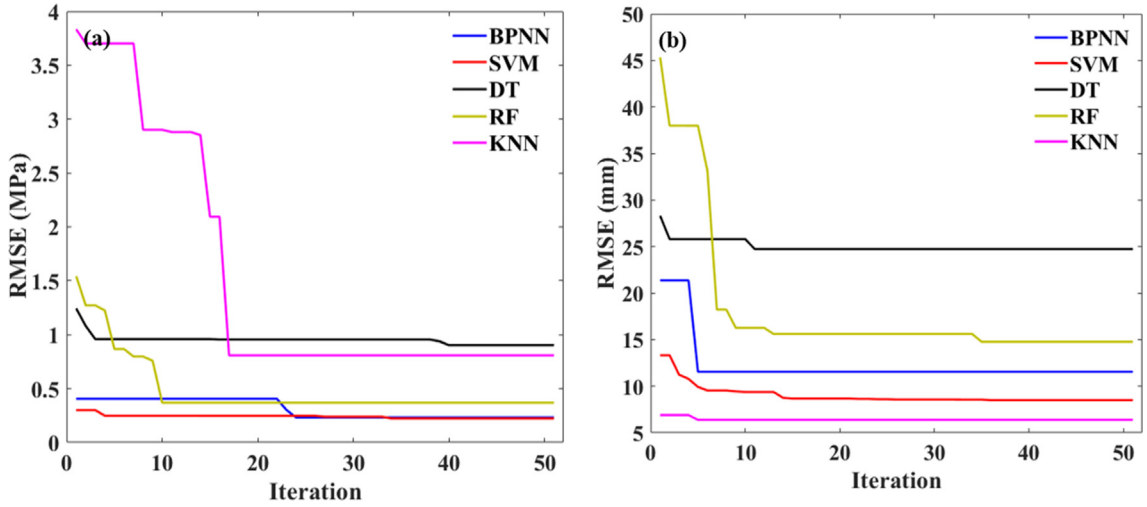


Fig. 15. Tuning hyperparameters of the ML models using PSO on the (a) UCS data set and (b) the slump data set for plastic concrete.

Table 10
Predition results of the ML algorithms for plastic concrete.

Data set	Model	Hyperparameter	RMSE		R	
			Training set	Test set	Training set	Test set
UCS (MPa)	BPNN	layer_num = 1 neuron_num = 11	0.48	0.66	0.99	**0.99**
	SVR	C = 36.7; γ = 3.7	0.51	1.0	0.99	0.99
	RT	min_samples_split = 11;min_samples_leaf = 1	2.1	2.2	0.80	0.86
	RF	min_samples_leaf = 1; tree_num = 12	1.7	2.5	0.88	0.88
	KNN	neighbor_num = 6	0.26	1.7	1.0	0.91
	LR	–	1.9	2.6	0.92	0.89
	MLR	–	1.9	2.3	0.84	0.86
Slump (mm)	BPNN	layer_num = 1 neuron_num = 8	29.8	43.9	0.80	0.58
	SVR	C = 16; γ = 19.6	45	42	0.69	0.67
	RT	min_samples_split = 8; min_samples_leaf = 1	43	46	0.48	0.26
	RF	min_samples_leaf = 1; tree_num = 28	11	15	0.98	**0.95**
	KNN	neighbor_num = 8	49	45	0.37	0.38
	LR	–	43	41	0.64	0.54
	MLR	–	40	42	0.59	0.47

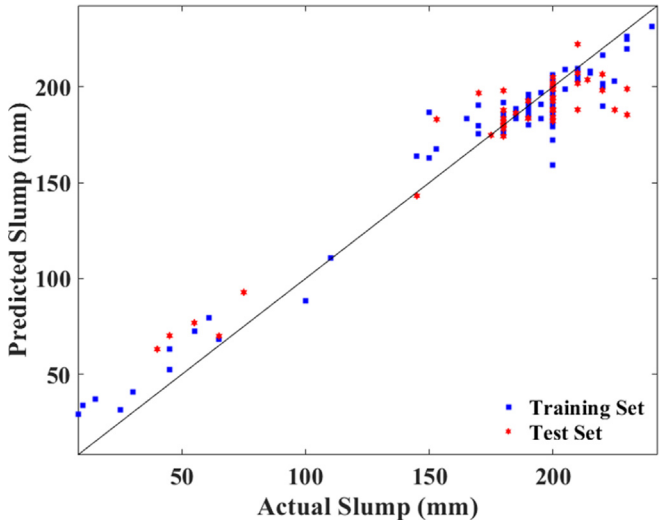


Fig. 16. Predicted versus actual values for slump prediction using RF for plastic concrete.

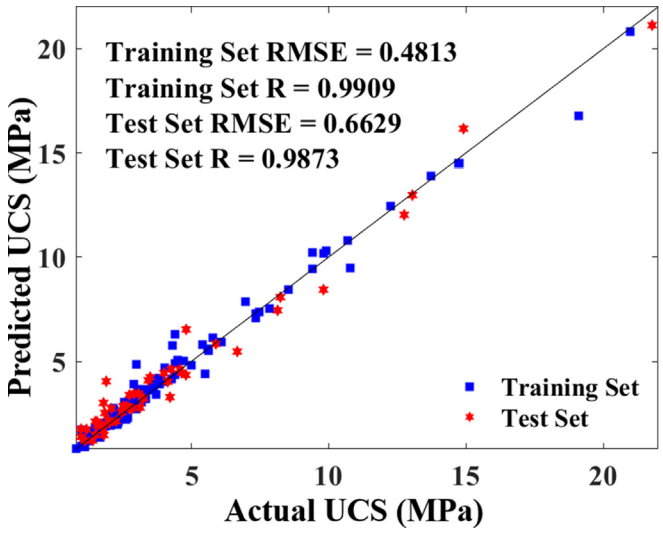


Fig. 17. Predicted versus actual values for UCS prediction using BPNN for plastic concrete.

an insignificant decrease in RMSE is observed. This indicates that the initial hyperparameter values are close to the optimal values. After convergence, the optimal hyperparameters are obtained (see Table 10). Then, the ML models with optimal hyperparameters are evaluated on the training and test sets, respectively. The prediction results are tabulated in Table 10. The highest R values on test set are typed in bold. For UCS, BPNN and SVR achieve the highest R value (0.99) on the test set. However, the overfitting ratio of SVR is higher (the training set RMSE divided by the test set RMSE) (Table 10). The scatter plot of predicted versus actual UCS values on training and test sets for BPNN are shown in Figure 17. As for slump prediction, RF obtains the highest prediction accuracy with the highest R (0.95) and the lowest RMSE (15 mm) on the test set. This is probably caused by the special structure of the slum data. The scatter plot of predicted versus actual slump values on training and test sets for RF are shown in Figure 16. It can be seen from Table 11 that the same slump value corresponds to different mixture proportions. RF can achieve higher prediction accuracy on imbalanced and more discrete slump data thanks to its bagging strategy [45]. In conclusion, BPNN has higher prediction accuracy on continuous data such as UCS (Fig. 18a), whereas RF behaves better on more discrete data such as slump (Fig. 18b). Therefore, BPNN and RF are used as the objective functions for UCS and slump of plastic concrete, respectively, in the optimization process. The generated BPNN and RF models are included in the Supplementary Materials.

3.2.4. Results of the tri-objective optimization for plastic concrete

For the tri-objective optimization problem, we aim to minimize the cost and maximize UCS and slump under the given constraints. Fig. 19 shows the Pareto front of the tri-objective optimization problem. In total, 200 nondominated points (optimal mixture proportions of plastic concrete) are obtained using RF and MOPSO. Each point is obtained based on the trade-off between the defined objectives. It is seen that points A, B, and C refer to single-objective optimization when UCS, cost, and slump are the sole objectives in the problem, respectively. As described earlier, a single optimal solution cannot be determined in a multi-objective optimization problem. With the use of the TOPSIS decision-making method, a more appropriate solution can be found. An ideal point that has the highest TOPSIS score of 1 has UCS = 3.5 MPa, slump = 210.3 mm, and cost = 34.2 $/m^3. The corresponding mixture proportions of these points on the Pareto front are tabulated in Table 12.

Table 11
Example of the data samples in the slump data set.

Gravel (kg.m^{-3})	Sand (kg.m^{-3})	Silty clay (kg.m^{-3})	Cement (kg.m^{-3})	Bentonite (kg.m^{-3})	Water (kg.m^{-3})	Slump (mm)
786	524	180	120	70	370	230
655	655	180	120	70	370	230
622	509	0	258	42.3	465	230
524	786	180	120	70	370	230
195	1305	0	200	50	420	230
467	571	0	274	44.8	493	225
0	955	0	289	47.3	520	225
795	705	0	220	40	400	220
795	705	0	220	39.6	400.4	220
750	750	0	200	30	500	220
750	750	0	200	40	500	220
651	651	180	120	100	348	220
450	900	150	120	16	300	220
195	1305	0	150	40	420	220

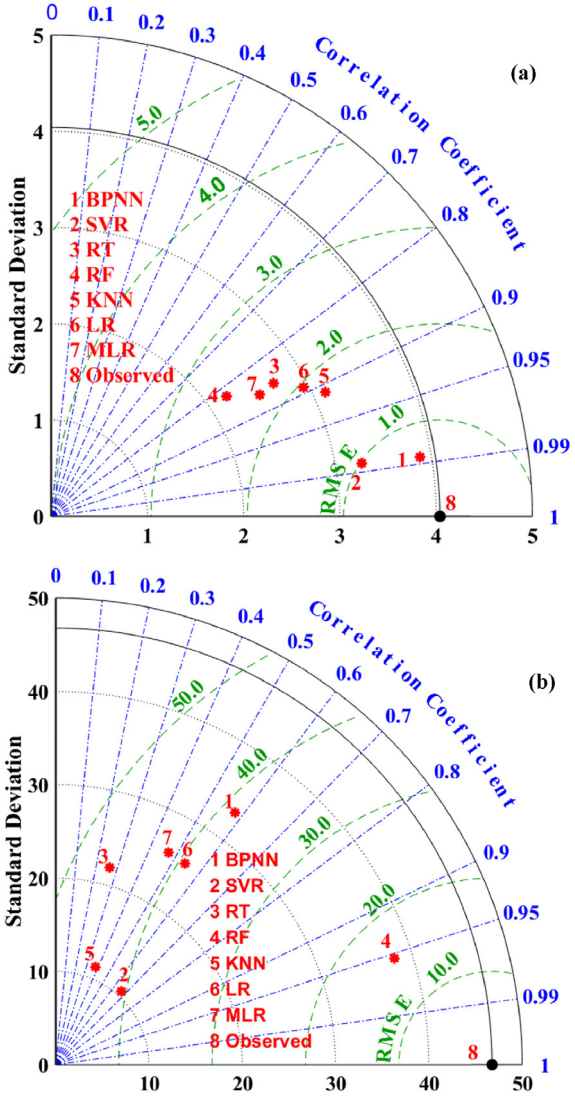


Fig. 18. Performance of the ML models on the test set of (a) UCS and (b) slump for plastic concrete.

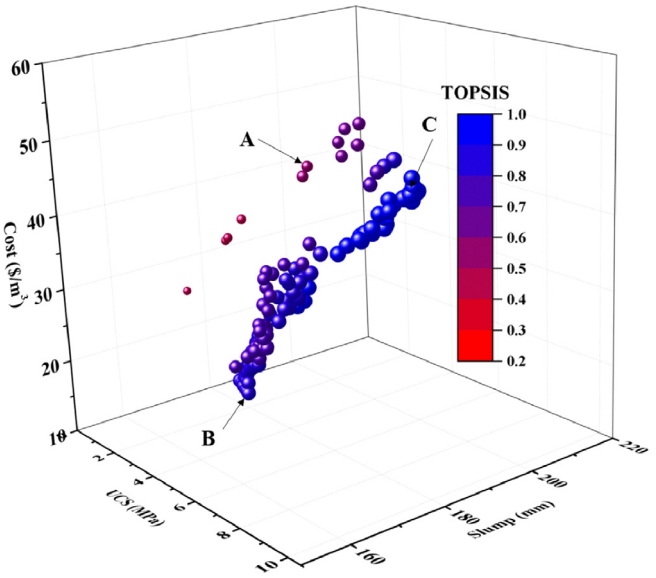


Fig. 19. Pareto front showing the trade-off among cost, UCS and slump for plastic concrete.

Table 12
Mixture proportions of different selected points on the Pareto front for plastic concrete.

Mixture	A	B	C	Final point
Gravel (kg/m^3)	233.8	52.9	110.3	92.3
Sand (kg/m^3)	1287.8	506.6	1058.7	1064.0
Silty clay (kg/m^3)	19.0	260	2.8	2.3
Cement (kg/m^3)	347.0	59.5	219.0	203.8
Bentonite (kg/m^3)	26.5	8.4	71.7	56.8
Water (kg/m^3)	284.1	520	452.1	469.4
UCS (MPa)	7.9	1.9	3.7	3.5
Slump (mm)	166.6	180.6	211.6	210.3
Cost ($/m^3)	50.4	11.8	37.2	34.2
TOPSIS score	0.55	0.83	0.99	1

4. Conclusion

Although optimization of concrete mixture proportions has been studied for several decades, laboratory-based experimental methods are still the main optimization methods. These methods are time- and resource-intensive and sometimes impossible to implement when multiple competing objectives need to be optimized and many influencing variables and highly nonlinear constraints need to be considered. Instead, multi-objective computational optimization methods based on machine learning and metaheuristic algorithms can automatically learn from a variety of experimental data and help design the mixture of concrete. In this study, a MOPSO model is used to balance the competing

objectives of concrete, with ML models as the objective functions. BPNN is found to have higher prediction accuracy on continuous UCS data, whereas RF performs better on more discrete slump data. The Pareto optimal solutions of the bi-objective and tri-objective optimization problems can be successfully obtained using the MOO approach, which can serve as a guide for the optimal design of concrete before the construction phase.

In future work, a highly accessible, continuously updated, and multinational database must be collected to increase the generalization ability of the MOO model for the mixture design of plastic concrete. In addition, we need to improve the prediction accuracy by incorporating other ML techniques, such as feature engineering and model comparison. In the final step, the MOO model needs to be incorporated into construction systems to facilitate the production of concrete during the early construction phase.

CRediT authorship contribution statement

Junfei Zhang: Conceptualization, Methodology, Software, Writing - original draft. **Yimiao Huang:** Data curation, Supervision, Writing - review & editing. **Yuhang Wang:** Data curation. **Guowei Ma:** Supervision.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

[1] M. DeRousseau, J. Kasprzyk, W. Srubar III, Computational design optimization of concrete mixtures: a review, Cem. Concr. Res. 109 (2018) 42–53.
[2] A. Behnood, V. Behnood, M. Modiri Gharehveran, K.E. Alyamac, Prediction of the compressive strength of normal and high-performance concretes using M5P model tree algorithm, Construction and Building Materials 142 (2017) 199-207.
[3] X.-S. Yang, Engineering Optimization: An Introduction with Metaheuristic Applications, John Wiley & Sons, 2010.
[4] M.-Y. Cheng, D. Prayogo, Y.-W. Wu, Novel genetic algorithm-based evolutionary support vector machine for optimizing high-performance concrete mixture, J. Comput. Civil Eng. 28 (4) (2014) 06014003.
[5] E.M. Golafshani, A. Behnood, Estimating the optimal mix design of silica fume concrete using biogeography-based programming, Cem. Concr. Compos. 96 (2019) 95–105.
[6] J.-H. Lee, Y.-S. Yoon, J.-H. Kim, A new heuristic algorithm for mix design of high-performance concrete, KSCE J. Civ. Eng. 16 (6) (2012) 974–979.
[7] I.C. Yeh, Computer-aided design for optimum concrete mixtures, Cem. Concr. Compos. 29 (3) (2007) 193–202.
[8] W. Gong, Z. Cai, L. Zhu, An efficient multiobjective differential evolution algorithm for engineering design, Struct. Multidiscip. Optim. 38 (2) (2009) 137–157.
[9] I.-C. Yeh, Optimization of concrete mix proportioning using a flattened simplex–centroid mixture design and neural networks, Eng. Comput. 25 (2) (2009) 179.
[10] J.L. McClelland, D.E. Rumelhart, G.E. Hinton, The Appeal of Parallel Distributed Processing, Morgan Kaufmann, San Mateo, CA, US, 1988.
[11] M. Dorofki, A.H. Elshafie, O. Jaafar, O.A. Karim, S. Mastura, Comparison of artificial neural network transfer functions abilities to simulate extreme runoff data, in: International Proceedings of Chemical, Biological and Environmental Engineering, 2012, pp. 39–44.
[12] C.J. Burges, A tutorial on support vector machines for pattern recognition, Data Min. Knowl. Disc. 2 (2) (1998) 121–167.
[13] D. Basak, S. Pal, D.C. Patranabis, Support vector regression, Neural Inf. Process.-Lett. Rev. 11 (10) (2007) 203–224.
[14] S. Boyd, L. Vandenberghe, Convex Optimization, Cambridge University Press, 2004.
[15] J.R. Quinlan, C4. 5: programs for machine learning, Elsevier, 2014.
[16] R.E. Schapire, The Boosting Approach to Machine Learning: An Overview, Nonlinear Estimation and Classification, Springer, 2003, pp. 149–171.
[17] S. Dhanabal, S. Chandramathi, A review of various k-nearest neighbor query processing techniques, Int. J. Comput. Appl. 31 (7) (2011) 14–22.
[18] D.W. Hosmer Jr, S. Lemeshow, R.X. Sturdivant, Applied Logistic Regression, John Wiley & Sons, 2013.
[19] R.H. Myers, R.H. Myers, Classical and modern regression with applications, Duxbury press Belmont, CA, 1990.
[20] J. Kennedy, Particle swarm optimization, Encyclopedia Mach. Learning (2010) 760–766.
[21] G.C. Cawley, N.L. Talbot, On over-fitting in model selection and subsequent selection bias in performance evaluation, Journal of Machine Learning Research 11(Jul) (2010) 2079-2107.
[22] C.-W. Hsu, C.-C. Chang, C.-J. Lin, A practical guide to support vector classification, (2003).
[23] R.J. Hyndman, A.B. Koehler, Another look at measures of forecast accuracy, Int. J. Forecast. 22 (4) (2006) 679–688.
[24] R. Boddy, G. Smith, Statistical Methods in Practice: For Scientists and Technologists, John Wiley & Sons, 2009.
[25] K. Deb, Multi-Objective Optimization Using Evolutionary Algorithms, John Wiley & Sons, 2001.
[26] G. Mavrotas, Effective implementation of the ε-constraint method in Multi-Objective Mathematical Programming problems, Appl. Math. Comput. 213 (2) (2009) 455–465.
[27] J.-h. Ryu, S. Kim, H. Wan, Pareto front approximation with adaptive weighted sum method in multiobjective simulation optimization, Winter Simulation Conference, Winter Simulation Conference, Austin, Texas, 2009, pp. 623-633.
[28] C.A.C. Coello, G.T. Pulido, M.S. Lechuga, Handling multiple objectives with particle swarm optimization, IEEE Trans. Evol. Comput. 8 (3) (2004) 256–279.
[29] K.P. Yoon, C.-L. Hwang, Multiple Attribute Decision Making: An Introduction, Sage Publications, 1995.
[30] I.-C. Yeh, Modeling of strength of high-performance concrete using artificial neural networks, Cem. Concr. Res. 28 (12) (1998) 1797–1808.
[31] M.I. Khan, Mix proportions for HPC incorporating multi-cementitious composites using artificial neural networks, Constr. Build. Mater. 28 (1) (2012) 14–20.
[32] W. Meng, M. Valipour, K.H. Khayat, Optimization and performance of cost-effective ultra-high performance concrete, Mater. Struct. 50 (1) (2017) 29.
[33] S.L. Garvin, C.S. Hayles, The chemical compatibility of cement–bentonite cut-off wall material, Constr. Build. Mater. 13 (6) (1999) 329–341.
[34] Y. Yu, J. Pu, K. Ugai, Study of mechanical properties of soil-cement mixture for a cutoff wall, Soils Found. 37 (4) (1997) 93–103.
[35] G. Fenoux, Filling materials for watertight cut off walls, Commission Internationale des grands barrages, 1985.
[36] P.K. Mehta, Concrete. Structure, properties and materials, (1986).
[37] A.T. Amlashi, S.M. Abdollahi, S. Goodarzi, A.R. Ghanizadeh, Soft computing based formulations for slump, compressive strength, and elastic modulus of bentonite plastic concrete, J. Cleaner Prod. 230 (2019) 1197–1216.
[38] A. Hajighasemi, Investigation of Allowable Hydraulic Gradient in Plastic Concrete, University of Tehran, Tehran, 1998.
[39] A. Mahboubi, A. Ajorloo, Experimental study of the mechanical behavior of plastic concrete in triaxial compression, Cem. Concr. Res. 35 (2) (2005) 412–419.
[40] A. Mahboubi, M. Anari, Effects of mixing proportions and sample age on mechanical properties of plastic concrete; an experimental study, The Proceedings of the 1st International Conference on Concrete Technology, Tabriz, 2009.
[41] A. Pashazadeh, M. Chekani-Azar, Estimating an appropriate plastic concrete mixing design for cutoff walls to control leakage under the earth dam, J. Basic Appl. Sci. Res. 1 (9) (2011) 1295–1299.
[42] A.R. Ghanizadeh, H. Abbaslou, A.T. Amlashi, P. Alidoust, Modeling of bentonite/sepiolite plastic concrete compressive strength using artificial neural network and support vector machine, Front. Struct. Civ. Eng. 13 (1) (2019) 215–239.
[43] L. Hu, D. Gao, Stress-strain Relation Model and Failure Criterion of Plastic Concrete Under Compression, Zhengzhou University, Zhengzhou, 2012.
[44] L. Qing-fu, Z. Peng, Experimental research on strength of plastic concrete, Concrete 5 (2006) 23.
[45] C. Chen, A. Liaw, L. Breiman, Using Random Forest to Learn Imbalanced Data 1–12, University of California, Berkeley, 2004, p. 24.
[46] Y. Sun, J. Zhang, G. Li, Y. Wang, J. Sun, C. Jiang, Optimized neural network using beetle antennae search for predicting the unconfined compressive strength of jet grouting coalcretes, International Journal for Numerical and Analytical Methods in Geomechanics 43 (4) (2019) 801–813.
[47] Y. Sun, J. Zhang, G. Li, G. Ma, Y. Huang, J. Sun, Y. Wang, B. Nener, Determination of Young's modulus of jet grouted coalcretes using an intelligent model, Engineering Geology 252 (2019) 43–53.
[48] J. Sun, J. Zhang, Y. Gu, Y. Huang, Y. Sun, G. Ma, Prediction of permeability and unconfined compressive strength of pervious concrete using evolved support vector regression, Construction and Building Materials 207 (2019) 440–449.
[49] J. Zhang, G. Ma, Y. Huang, J. Sun, F. Aslani, B. Nener, Modelling uniaxial compressive strength of lightweight self-compacting concrete using random forest regression, Construction and Building Materials 210 (2019) 713–719.
[50] J. Zhang, D. Li, Y. Wang, Predicting uniaxial compressive strength of oil palm shell concrete using a hybrid artificial intelligence model, Journal of Building Engineering 30 (2020).
[51] J. Zhang, D. Li, Y. Wang, Toward intelligent construction: Prediction of mechanical properties of manufactured-sand concrete using tree-based models, Journal of Cleaner Production 258 (2020) 120665.
[52] J. Zhang, D. Li, Y. Wang, Predicting tunnel squeezing using a hybrid classifier ensemble with incomplete data, Bulletin of Engineering Geology and the Environment (2020) 1–12.
[53] J. Zhang, Y. Huang, G. Ma, B. Nener, Multi-objective beetle antennae search algorithm, arXiv Preprint (2020).